

# Git & GitHub



M<sup>a</sup> Dolores Rodríguez Moreno



Universidad  
de Alcalá



ISG

# Objectives

## Specific Objectives

- Understand what Git and GitHub are and how they differ
- Connect a local repository to GitHub and share your code online
- Review and track changes

## Sources

- Mitchell Wand: A Simple Introduction to Git: a distributed version-control system
- Shrija Sheth: GIT & GITLAB - STARTERKIT

# Outline

- Introduction
- Getting Started
- Key Concepts in Git
- Local vs Remote Repositories
- Git Project Lifecycle
- Basic Git Commands
- Intermediate Git Commands
- Working with GitHub
- Common Git Command Sequence
- Conclusions



# Introduction

- What is Version Control?
  - A system that records changes to files over time
  - It helps manage code and collaborate safely on the same project
- Why use Version Control?
  - Prevents loss of work and keeps project history
  - Enables collaboration with other developers
  - Lets you experiment safely using branches
  - Makes sharing and publishing your work easy
  - No more 'final\_final\_v3.py' chaos!

# What is Git?

- Git is a version control system created by Linus Torvalds
- It tracks changes in your project's files over time
- You can go back to previous versions if something breaks
- It helps multiple people work on the same project safely
- Works locally (no Internet required)



# What is GitHub?

- GitHub is an online platform for hosting Git repositories
- It enables collaboration between developers
- Built on top of Git, but adds features like web interface, access control, pull requests, issues, and wikis
- Example: Your Computer  $\rightleftarrows$  GitHub  $\rightleftarrows$  Teammates



# Git vs GitHub

- Git is a local program for version control
- GitHub is an online service for hosting and collaboration
- Git works offline; GitHub requires Internet
- Example Git command: `git commit -m 'message'`
- Example GitHub command: `git push origin main`

# Outline

- Introduction
- **Getting Started**
- Key Concepts in Git
- Local vs Remote Repositories
- Git Project Lifecycle
- Basic Git Commands
- Intermediate Git Commands
- Working with GitHub
- Common Git Command Sequence
- Conclusions



# Getting Started

- Install Git in your machine (Windows/Linux/MAC)
  - git --version
  - git config -h (list of commands)
  - git config --help (git manual)
- Set your user Git
  - git config --global user.name
- Set your e-mail for Git
  - git config --global user.email
- DO NOT CONTINUE until the 3 steps are verified

# Outline

- Introduction
- Getting Started
- **Key Concepts in Git**
- Local vs Remote Repositories
- Git Project Lifecycle
- Basic Git Commands
- Intermediate Git Commands
- Working with GitHub
- Common Git Command Sequence
- Conclusions

# Key Concepts in Git

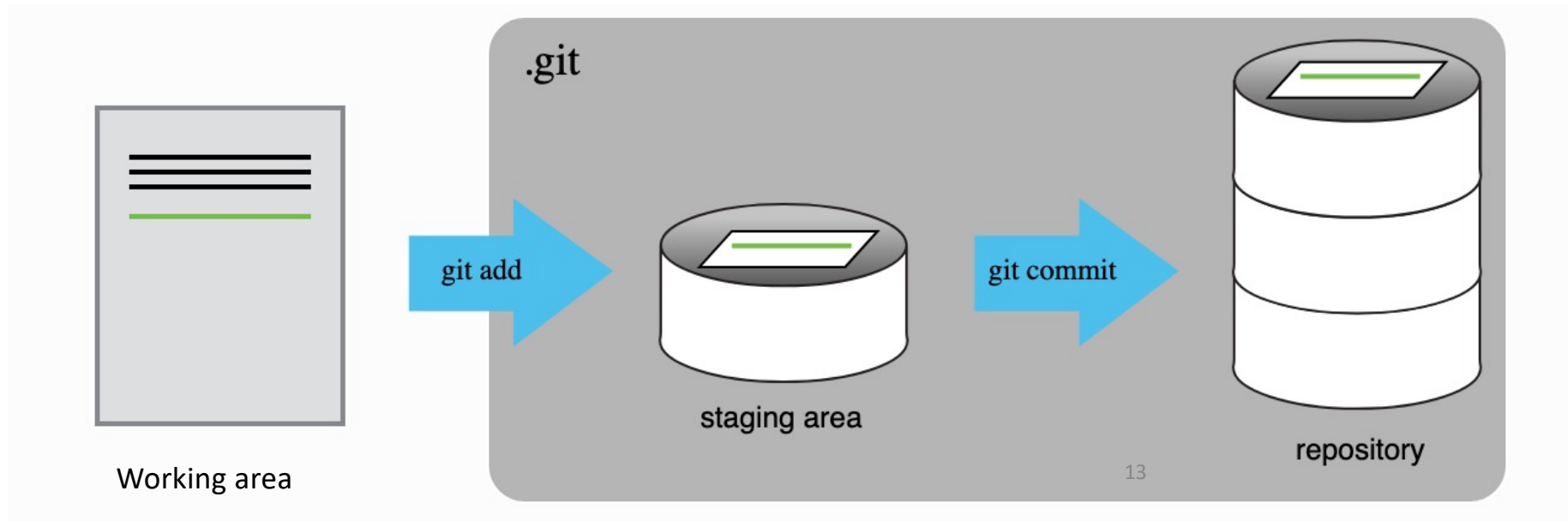
- Repository: Folder tracked by Git with full history
- Commit: Snapshot of your project at a point in time
- Branch: Independent line of development
- Merge: Combines changes from one branch into another
- HEAD: Pointer to your current branch or commit

# Basic Git Workflow

- Git has three areas: Working Directory, Staging Area, Repository
- Typical process:
  1. Edit files
  2. `git add` – stage your changes
  3. `git commit` – save a snapshot
  4. `git push` – upload (optional)

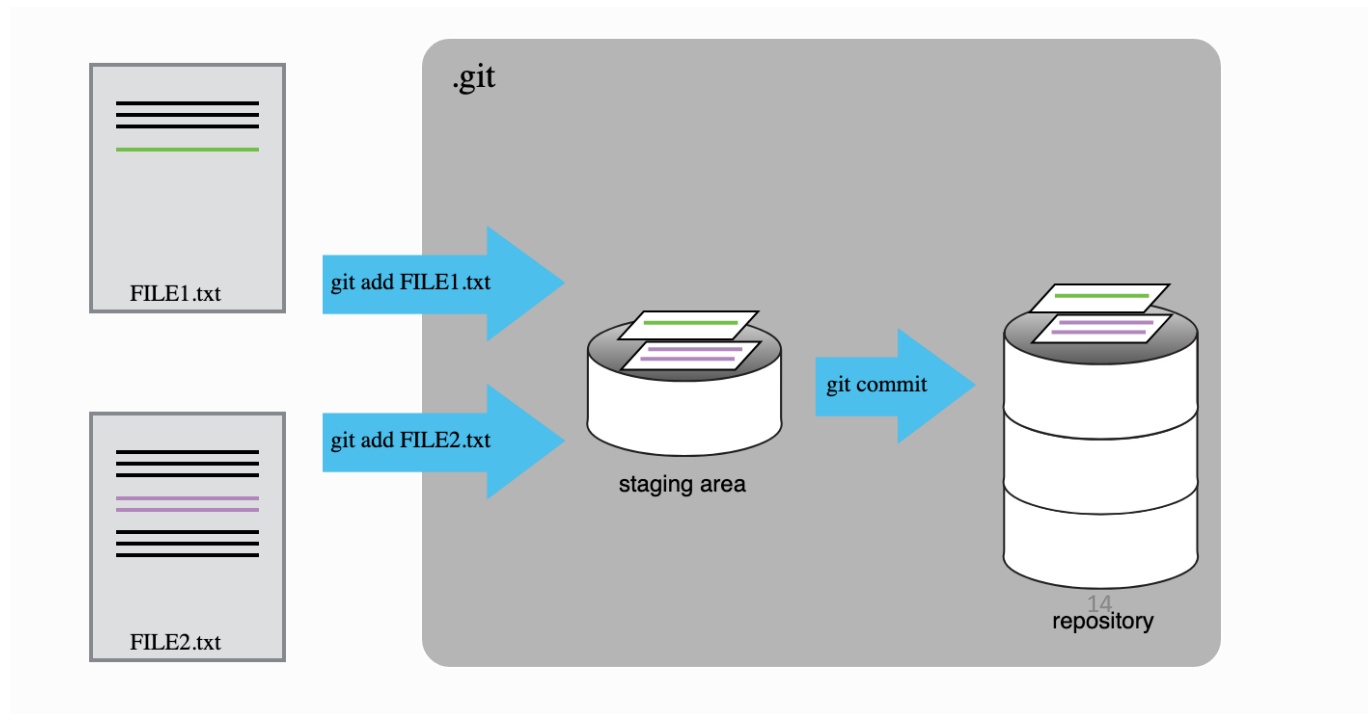


# Basic Git Workflow



Source: Shrija Sh

# Basic Git Workflow



Source: Shrija Sh

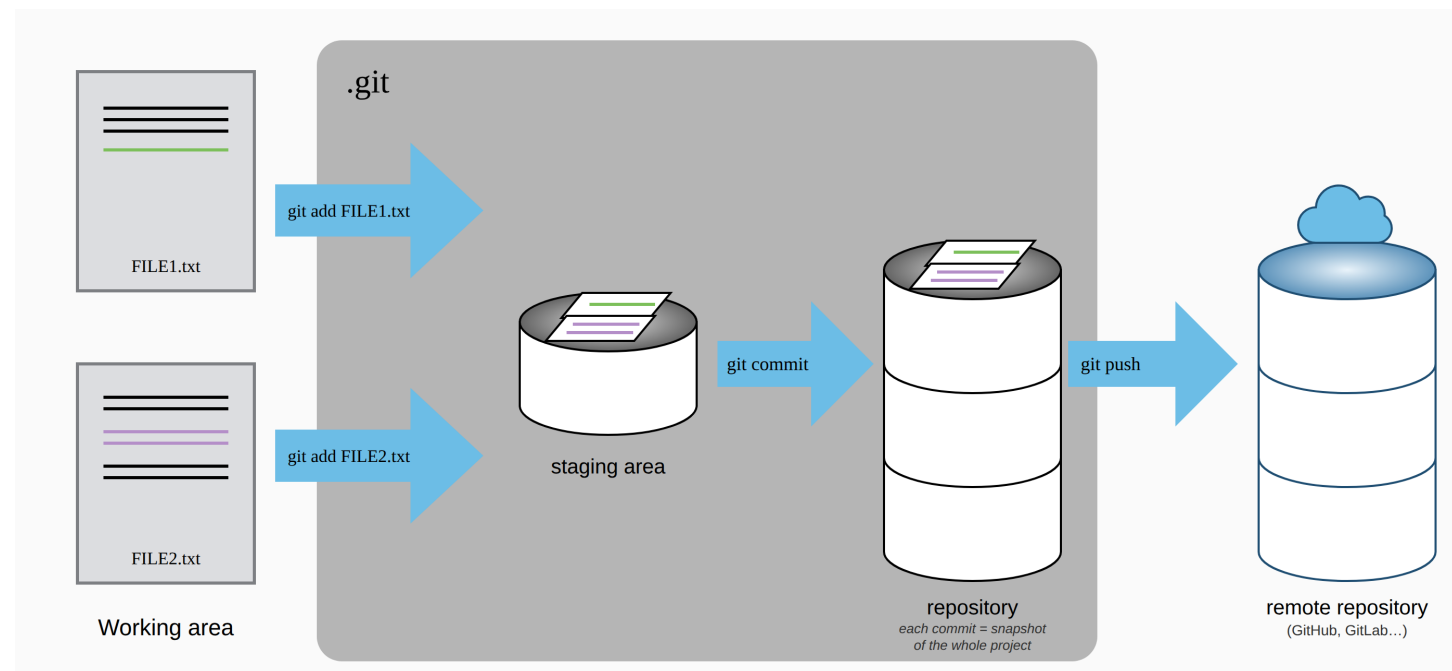
# Outline

- Introduction
- Getting Started
- Key Concepts in Git
- **Local vs Remote Repositories**
- Git Project Lifecycle
- Basic Git Commands
- Intermediate Git Commands
- Working with GitHub
- Common Git Command Sequence
- Conclusions

# Local vs Remote Repositories

- Local repo: lives on your computer, works offline
- Remote repo: hosted on GitHub or similar, for collaboration
- Common commands:
  - `git push origin main`
  - `git pull origin main`
- Git connects them using 'remotes' (usually named origin)

# GitHub Workflow



# Outline

- Introduction
- Getting Started
- Key Concepts in Git
- Local vs Remote Repositories
- **Git Project Lifecycle**
- Basic Git Commands
- Intermediate Git Commands
- Working with GitHub
- Common Git Command Sequence
- Conclusions

# Git Project Lifecycle

1. Create or clone a repository
2. Initialize Git with 'git init'
3. Add files with 'git add'
4. Commit changes with 'git commit'
5. Push your work online with 'git push'

# Outline

- Introduction
- Getting Started
- Key Concepts in Git
- Local vs Remote Repositories
- Git Project Lifecycle
- **Basic Git Commands**
- Intermediate Git Commands
- Working with GitHub
- Common Git Command Sequence
- Conclusions

# Basic Git Commands

- git init – Start a new Git repository in a folder
- git add – Add files to the staging area
- git commit – Save a snapshot of your changes
- git log – Show the commit history
- git config – Set your username and email

# Outline

- Introduction
- Getting Started
- Key Concepts in Git
- Local vs Remote Repositories
- Git Project Lifecycle
- Basic Git Commands
- **Intermediate Git Commands**
- Working with GitHub
- Common Git Command Sequence
- Conclusions

# Intermediate Git Commands

- git diff – See what's changed since the last commit
- git reset – Unstage or undo changes
- git checkout – Switch branches or restore files
- git branch – List or create branches
- git merge – Combine changes from one branch into another

# Track Changes

- `git status` → which files are modified, staged or untracked
- `git log` → list of commits: ID, author, date and message
- `git log --patch` → same list + the changes each commit introduced
- `git diff` → changes NOT yet staged  
(working area vs staging area)
- `git diff --staged` → changes staged for the next commit  
(staging area vs last commit)
- Tip: long output opens in a viewer — press q to quit

# Committing a folder

- `mkdir temp`
- `ls -a`
- `git status`
- `touch temp/.gitkeep` (Create an empty file inside an empty folder)
- `git status`
- `git add .`
- `git commit -m "Added a temp folder"`
- `git status`

# Delete Files

- touch newfile.txt
- ls -a
- git status
- rm newfile.txt
- git status
- Question: Will it be removed?



# Delete Folder

- `rm -rf -- temp`
- `ls -a`
- `git status`
- `git add .`
- `git commit -m "removed file and folder"`
- Question: Will it be removed?



# Ignore files (For private files not in repo)

- `mkdir config`
- `touch config/private.txt`
- `git status`
- `vim .gitignore` (config/ - specify files and folders you want to ignore)
- `git status`
- `ls -a`
- `git add .`
- `git commit -m "Added gitignore config"`
- `git status`
- `git status --ignored`

# Outline

- Introduction
- Getting Started
- Key Concepts in Git
- Local vs Remote Repositories
- Git Project Lifecycle
- Basic Git Commands
- Intermediate Git Commands
- **Working with GitHub**
- Common Git Command Sequence
- Conclusions

# Working with GitHub (Remote Commands)

- `git remote add origin <URL>`: connect local repo to GitHub
- `git clone`: copy a repo from GitHub to your computer
- `git push`: upload commits to GitHub
- `git pull`: download updates from GitHub
- Typical flow: Local repo  $\rightleftharpoons$  (push/pull)  $\rightleftharpoons$  GitHub

# Outline

- Introduction
- Getting Started
- Key Concepts in Git
- Local vs Remote Repositories
- Git Project Lifecycle
- Basic Git Commands
- Intermediate Git Commands
- Working with GitHub
- **Common Git Command Sequence: From Local to GitHub**
- Examples
- Conclusions

# Initial Workflow

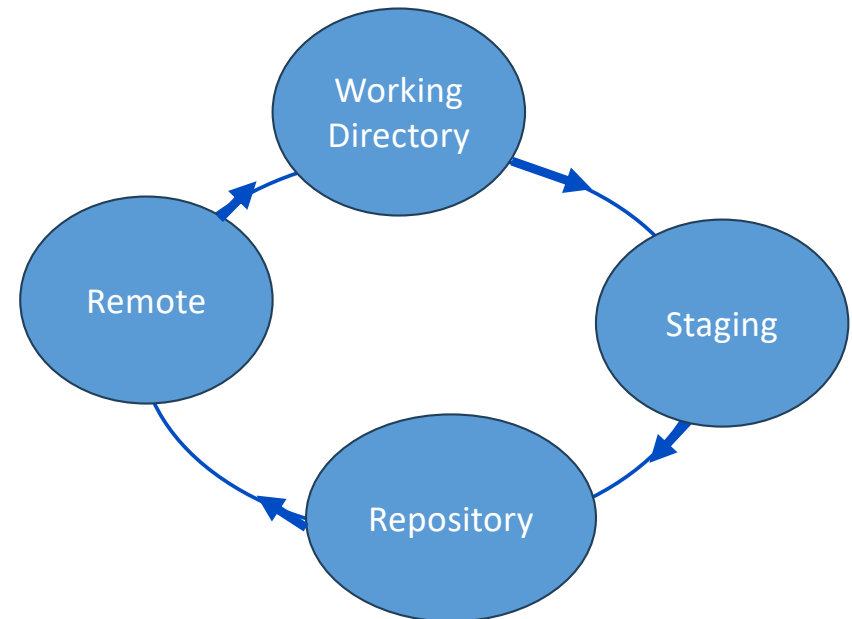
1. `git init` – Initialize Git
2. `git add .` – Stage all files
3. `git commit -m 'Initial commit'` – Save snapshot with a message
4. `git push -u origin main` – Upload your work to GitHub
5.  Edit → **+** Add →  Commit →  Push

# Outline

- Introduction
- Getting Started
- Key Concepts in Git
- Local vs Remote Repositories
- Git Project Lifecycle
- Basic Git Commands
- Intermediate Git Commands
- Working with GitHub
- Common Git Command Sequence
- **Conclusions**

## Steps summary

- Tip: Always git pull before starting new work to avoid conflicts.
- Edit your files locally
- Add them to the staging area (git add)
- Commit to save a version (git commit)
- Push to share with others (git push)
- Pull to update your work (git pull)



# Good Practices

- Encourage clean, professional Git habits
-  Make frequent, small commits (each one should do one clear thing)
-  Write clear commit messages, e.g. `git commit -m "Fix login bug"`
-  Use branches for each new feature or experiment
-  Pull regularly to stay up-to-date
-  Clean up unused branches after merging
-  Never commit sensitive data (passwords, tokens)
-  Review your history with `git log` before pushing

# Tips for Collaboration

- Create a clear *README.md* for your project
- Use Pull Requests instead of pushing to main directly
- Review teammates' code before merging
- Tag important versions with `git tag v1.0`